AI Lab: Informed Search Techniques in Python

CLO - 2: *Apply classical AI techniques, including uninformed and informed search algorithms, minimax, and neural networks, to solve structured problems.*

Lab Objectives:

1. Understand heuristic functions and their role in informed search strategies.
2. Implement **Greedy Best-First Search (GBFS)** and **A* Search** on a weighted graph.
3. Evaluate and compare search strategies in terms of path optimality and computational efficiency.

Note: In all tasks, visualize each graph with the **solution path highlight** using **networkX** Library.

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb file, Github repo link(streamlit code), deployed App Link** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the total marks.**

Task 1: Designing a Heuristic for a Warehouse Robot

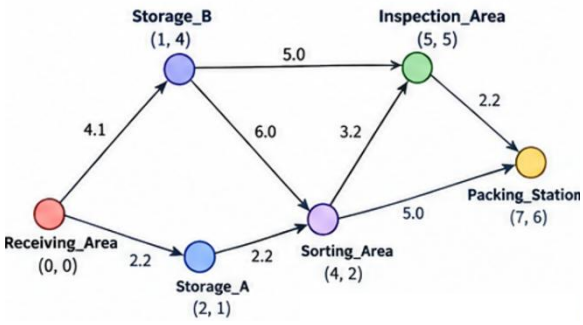
Problem Instance

An autonomous robot is operating inside a warehouse and needs to move a package from the **Receiving Area** to the **Packing Station**. The warehouse contains several interconnected locations, and the robot can move between connected locations.

The warehouse is represented as a weighted graph:

- A **node** represents a warehouse location.
- An **edge** represents a possible path between two locations.
- The **edge cost** represents the movement cost between two locations.
- The **start node** is the Receiving Area.
- The **goal node** is the Packing Station.

The approximate coordinates of the warehouse locations are:



Instructions

1. Represent the warehouse as a weighted graph in Python.
2. Store the coordinates of all locations.
3. Implement a heuristic function h(n) using the Euclidean distance between the current location and the Packing Station.
4. Calculate and display the heuristic value for each location.
5. Explain why Euclidean distance is a suitable heuristic for this problem.
6. Verify the heuristic condition for the given edges:

$$h(n) \leq cost(n,m) + h(m)$$

Deliverables

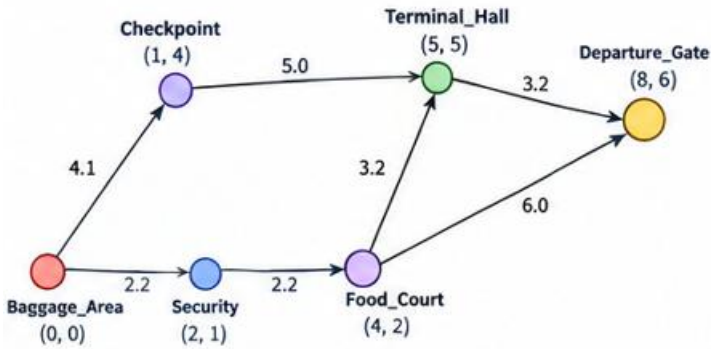
- Python implementation of the warehouse graph.
- Python implementation of the heuristic function.
- NetworkX visualization of the warehouse graph.
- Table showing the heuristic value of each location.
- Short explanation of why the selected heuristic is appropriate.

Task 2: Greedy Best-First Search for Airport Baggage Handling

Problem Instance:

An airport uses an automated baggage cart to transport luggage from the **Baggage Handling Area** to a passenger's **Departure Gate**. The cart can travel through several airport locations, but different paths have different movement costs.

The airport layout is represented using the following coordinates:



Instructions

1. Represent the airport layout as a weighted graph.
2. Use Euclidean distance to the Departure Gate as the heuristic.
3. Implement **Greedy Best-First Search (GBFS)** using:

f(n) = h(n)

4. Set:
    - Start node = Baggage_Area
    - Goal node = Departure_Gate
5. Record the order in which nodes are expanded.
6. Display the solution path found by GBFS.
7. Calculate the actual total cost of the path found.
8. Explain why GBFS selected the nodes in that order.

Deliverables

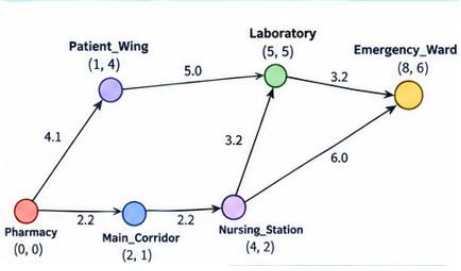
- Python implementation of GBFS.
- NetworkX visualization of the airport graph.
- Node expansion sequence.
- Solution path.
- Total path cost.
- Short explanation of how the heuristic guided GBFS.

Task 3: A* Search Algorithm for an Emergency Supply Robot

Problem Instance:

A hospital uses an autonomous robot to deliver an emergency medical supply from the **Pharmacy** to the **Emergency Ward**. The robot can use several corridors, and each corridor has a different movement cost.

The hospital locations have the following approximate coordinates:



Instructions

1. Represent the hospital corridors as a weighted graph.
2. Implement the Euclidean heuristic to estimate the remaining distance to the Emergency Ward.
3. Implement *A Search** using:

$$f(n) = g(n) + h(n)$$

4. Set:
    - Start node = Pharmacy
    - Goal node = Emergency_Ward
5. For each selected node, determine:
    - g(n) = actual cost from Pharmacy to the current node.
    - h(n) = estimated remaining cost to Emergency Ward.
    - $f(n) = g(n) + h(n)$
6. Record the order in which nodes are expanded.
7. Display the final path and its total cost.
8. Explain how considering g(n) changes the search behavior compared with GBFS.

Deliverables

- Python implementation of A*.
- NetworkX visualization of the hospital graph.
- Node expansion sequence.
- Solution path.
- Total path cost.
- A table showing g(n), h(n), and f(n) for the selected nodes.
- Short comparison of the search behavior of A* and GBFS.

GUI REUIREMENT: Interactive Search Visualization Using Streamlit

Create a **Graphical User Interface (GUI)** using **Streamlit** that allows the user to:

- Select an **initial node** from a dropdown menu.

- Select a **goal node** from a dropdown menu.
- Select a **Search Algorithm** from a dropdown menu: **GBFS** or **A***.
- Run the selected search algorithm on the given NetworkX graph.
- Display the **Network Graph** using the NetworkX library.
- Highlight the **solution path** returned by the selected algorithm on the graph.
- Display the **selected algorithm, solution path, and total path cost** below the graph.

The GUI should use the **same graph and search algorithms implemented in the previous tasks**. The purpose of this task is to integrate the GBFS/A* implementations with a user interface and visualize their **resulting solution paths**.

To turn the algorithms into a small practical application.

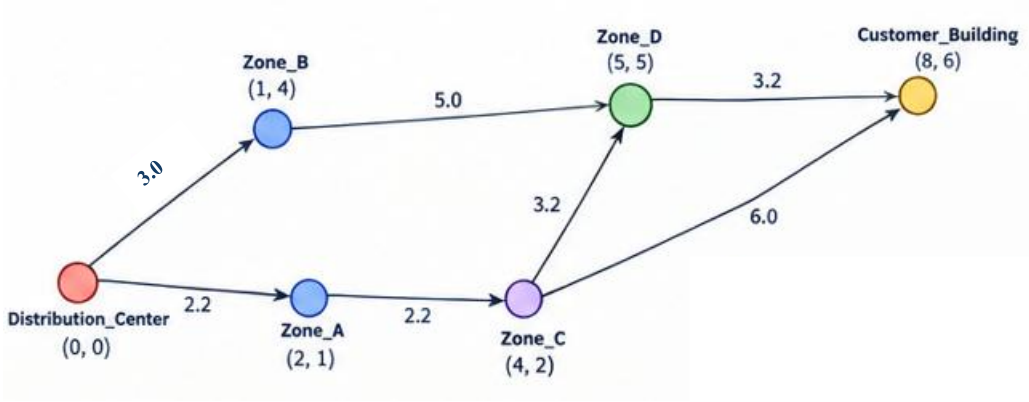
Task 4: Weighted A* for an Autonomous Delivery Drone

Problem Instance:

Modify A* evaluation function:

$$f(n) = g(n) + w \cdot h(n), w = 1.0,1.5,2,3$$

An autonomous delivery drone needs to transport a package from a **Distribution Center** to a **Customer Building**. The drone can travel through several designated locations, and each route has a different movement cost.
The company wants to investigate how giving greater importance to the heuristic affects **search behavior, number of nodes expanded, and solution cost**.
Use the following locations and coordinates:



Instructions

1. Implement normal A* using:

$$f(n) = g(n) + h(n)$$

2. Implement Weighted A* using:

$$f(n) = g(n) + w\,h(n)$$

3. Use Euclidean distance as the heuristic.
4. Run the search for:

$$w = 1,\ 1.5,\ 2,\ 3$$

5. For each value of w, record:
   - Solution path
   - Total path cost
   - Number of nodes expanded
6. Complete the following table:

Weight w	**Solution Path**	**Total Cost**	**Nodes Expanded**
1			
1.5			
2			
3			

7. Compare the results for different values of w.
8. Explain how increasing $w$changes the balance between:
   - actual cost $g(n)$
   - estimated cost $h(n)$
9. Explain why Weighted A* with $w > 1$may return a higher-cost path than normal A*.

Normal A*:

"Balance actual cost and estimated cost."

Weighted A*:

"Trust heuristic more than actual cost."

Does Weighted A* Guarantee Optimality?

- If w = 1 → Yes (normal A*)
- If w > 1 → No (may lose optimality)

But:

*Increasing **w** gives greater importance to the heuristic. This can reduce the amount of search in some problems, but it can also produce a higher-cost solution because $w > 1$ does not provide the same optimality guarantee as normal A.**

Task 5: Finding a Charging Station Using Multiple Goals

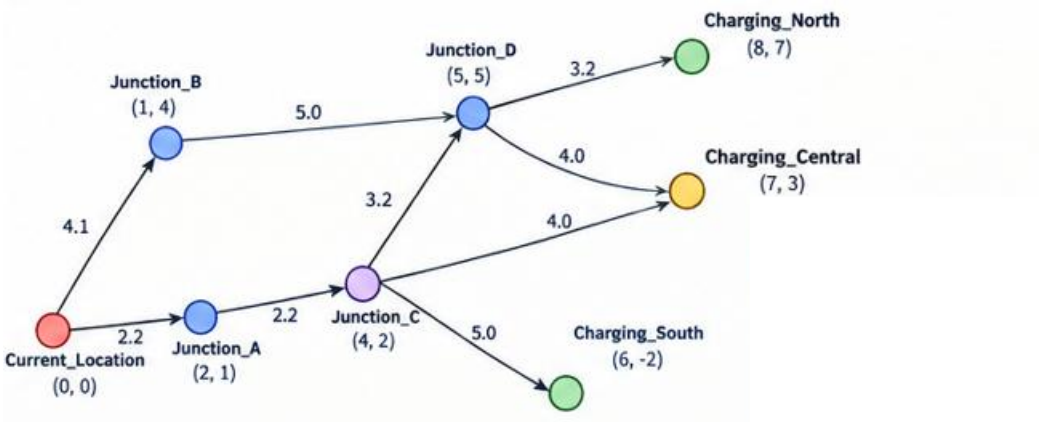
Problem Instance

An electric vehicle is travelling through a city and its battery level has become low. The vehicle needs to reach **any available charging station**. There are three possible charging stations:

- Charging_North
- Charging_Central
- Charging_South

The navigation system must find a path from the current location to one of these charging stations.

The locations are represented using the following coordinates:



Part A: Greedy Best-First Search

1. Implement GBFS using:

f(n) = h(n)

2. The heuristic should estimate the distance from the current location to the **nearest charging station**.
3. Start the search from Current_Location.
4. Stop when any charging station is reached.
5. Record:

- Node expansion order.
- Charging station reached.
- Solution path.
- Total path cost.

Part B: A* Search

1. Implement A* using:

$f(n) = g(n) + h(n)$

2. Use the same starting location and charging stations.
3. The search should find the lowest-cost path to any available charging station.
4. Record:
    - Node expansion order.
    - Charging station reached.
    - Solution path.
    - Total path cost.

Part C: Comparison

Compare GBFS and A* using the following table:

Measure	**GBFS**	**A***
Charging station reached		
Solution path		
Total path cost		
Number of nodes expanded		

Briefly explain why the two algorithms may make different search decisions even though they use the same heuristic information.

Deliverables

- Python implementation of GBFS.
- Python implementation of A*.
- NetworkX visualization of the charging network.

- Expansion sequence for both algorithms.
- Solution path and total cost for both algorithms.
- Completed comparison table.
- Short analysis of the results.

Ungraded Tasks:

Task 6: Heuristic Function Evaluation on 8-Puzzle

Problem Instance:

Initial state (3×3 puzzle):

1	2	3
4	0	5
6	7	8

Goal state:

1	2	3
4	5	6
7	8	0

Instructions:

1. Implement the following heuristics:
    - **h1(n):** Number of misplaced tiles.
    - **h2(n):** Manhattan distance (sum of distances of each tile from its goal position).
2. Compute h1 and h2 for the initial state and two intermediate states (you may select two states resulting from a single legal move).
3. Verify which heuristics are admissible and consistent.

Deliverables:

- Python functions for h1 and h2.
- Table showing heuristic values for initial and two intermediate states.
- Short explanation on admissibility and consistency.

Task 7: Greedy Best-First Search (GBFS) on a Weighted Grid

Problem Instance:

Consider a 4×4 grid with the following cell costs:

S	**1**	**4**	**7**
2	X	1	3
3	2	2	1
4	3	1	G

- **S** = Start (0,0)
- **G** = Goal (3,3)
- **X** = Obstacle (cannot pass)

Heuristic: Manhattan distance to Goal.

Instructions:

1. Implement **GBFS** using the given grid and heuristic.
2. Show the order of nodes expanded.
3. Compute the path found and its total cost.
4. Compare the result with BFS (uninformed search).

Deliverables:

- Python implementation of GBFS.
- Printed node expansion order and final path with total cost.
- Short comparison with BFS.